

并行程序设计与算法实验五报告

基于 OpenMP 与 Pthreads 的并行矩阵乘法

姓名：元朗曦 学号：23336294

2026 年 5 月 6 日

1 实验目的

本实验围绕矩阵乘法这一典型计算密集型问题，分别使用 OpenMP 与 Pthreads 进行并行实现，并分析线程数量、矩阵规模和调度策略对程序性能的影响。实验目的如下：

1. 掌握 OpenMP 并行循环的基本使用方法，理解 `parallel for` 与 `schedule` 子句的作用。
2. 实现基于 Pthreads 的 `parallel_for` 抽象，模拟 OpenMP 对并行循环的分解、分配与执行机制。
3. 使用矩阵乘法验证并行程序的正确性，掌握通过串行程序进行结果校验的方法。
4. 通过实验数据比较不同线程数、不同调度策略下的运行时间和 GFLOPS，分析并行加速效果。

2 实验原理

矩阵乘法

设矩阵 $A \in \mathbb{R}^{m \times n}$ ，矩阵 $B \in \mathbb{R}^{n \times k}$ ，矩阵乘法结果为 $C \in \mathbb{R}^{m \times k}$ ，则：

$$C_{ij} = \sum_{p=0}^{n-1} A_{ip} B_{pj}.$$

矩阵 C 的每一行可以独立计算，因此可以将外层行循环分配给多个线程。不同线程写入 C 的不同行，不存在写冲突，不需要加锁。

矩阵乘法的浮点运算量近似为：

$$\text{FLOPs} = 2mnk,$$

其中一次乘法和一次加法合计为 2 次浮点运算。程序性能使用 GFLOPS 衡量：

$$\text{GFLOPS} = \frac{2mnk}{t \times 10^9},$$

其中 t 为运行时间，单位为秒。

OpenMP 并行循环

OpenMP 是共享内存并行编程接口，使用编译指导语句描述并行区域。在本实验中，OpenMP 版本使用：

```
1 #pragma omp parallel for schedule(static)
2 for (int i = 0; i < m; ++i) {
3     // compute row i of C
4 }
```

调度策略包括：

- **默认调度**：由 OpenMP 运行时和编译器决定循环分配方式。
- **静态调度**：在循环开始前将迭代均匀划分给各线程，调度开销小。
- **动态调度**：运行时动态分配迭代，负载均衡能力更强，但同步开销更高。

Pthreads parallel_for

Pthreads 是较底层的线程库，需要显式创建、传参和回收线程。为了模拟 OpenMP 的并行循环，本实验实现了如下形式的 parallel_for：

```
1 int parallel_for(int start, int end, int inc,
2                 void* (*functor)(int, void*),
3                 void* arg,
4                 int num_threads,
5                 Schedule schedule);
```

其中 start、end 和 inc 描述循环范围，functor 表示每次迭代执行的函数，arg 传递共享参数，num_threads 表示期望线程数。静态调度将迭代区间平均分配给各线程；动态调度使用原子计数器为线程分配下一次迭代。

3 实验内容

本实验代码位于 lab/05/src，使用 CMake 作为构建工具。主要程序如下：

1. openmp_matmul.cpp：基于 OpenMP 的并行矩阵乘法。

2. `pthread_parallel_for_matmul.cpp`: 基于 Pthreads 的 `parallel_for`, 并以矩阵乘法作为测试样例。
3. `benchmark.py`: 随机生成矩阵规模并批量运行不同实现, 用于性能测试。

OpenMP 程序设计

OpenMP 程序从命令行读取矩阵规模、线程数和调度策略, 随机生成矩阵 A 和 B , 并计算矩阵 C 。程序支持如下参数:

```
1 ./openmp_matmul <m> <n> <k> <threads> [default|static|dynamic] [--verify]
   [--print]
```

其中矩阵规模 m, n, k 的范围为 $[128, 2048]$, 线程数量范围为 $[1, 16]$ 。--verify 选项会执行串行矩阵乘法并比较最大绝对误差; --print 选项用于输出矩阵 A, B, C 。

Pthreads 程序设计

Pthreads 程序首先实现通用的 `parallel_for`。随后将矩阵乘法的行计算封装为 functor:

```
1 void* matmul_row_functor(int row, void* raw_args) {
2     // compute one row of C
3     return nullptr;
4 }
5
6 parallel_for(0, m, 1, matmul_row_functor, &args, threads, schedule);
```

该设计将循环调度逻辑与具体计算逻辑分离, 使 `parallel_for` 可以复用于其他按循环迭代划分的任务。

Benchmark 流程

实验 benchmark 按如下流程执行:

1. 在 $[128, 2048]$ 范围内随机生成整数 m, n, k ;
2. 程序根据 m, n, k 随机生成矩阵 $A \in \mathbb{R}^{m \times n}$ 和 $B \in \mathbb{R}^{n \times k}$;
3. 调用并行矩阵乘法计算 $C = A \times B$;
4. 记录运行时间和 GFLOPS;
5. 重复选取不同随机规模, 分析矩阵规模变化对性能的影响。

benchmark 脚本支持选择实现、调度策略、样本数量和线程数。例如：

```
1 python3 lab/05/benchmark.py \  
2   --samples 4 \  
3   --program both \  
4   --schedules static \  
5   --threads 4 \  
6   --csv report/random_benchmark.csv
```

构建与运行

构建命令如下：

```
1 cmake -S lab/05 -B lab/05/build -DCMAKE_BUILD_TYPE=Release  
2 cmake --build lab/05/build -j
```

单次运行示例：

```
1 ./lab/05/build/openmp_matmul 512 512 512 4 static --verify  
2 ./lab/05/build/pthread_parallel_for_matmul 512 512 512 4 static --verify
```

4 实验结果

正确性验证

OpenMP 与 Pthreads 两个程序均通过串行矩阵乘法校验。以 $128 \times 128 \times 128$ 矩阵为例：

```
1 Task: OpenMP matrix multiplication  
2 A: 128 x 128  
3 B: 128 x 128  
4 C: 128 x 128  
5 Threads: 4  
6 Schedule: dynamic  
7 Time (sec): 0.000432  
8 GFLOPS: 9.699  
9 Verify: PASS, max_abs_diff=0.0000000000  
10  
11 Task: Pthreads parallel_for matrix multiplication  
12 A: 128 x 128  
13 B: 128 x 128  
14 C: 128 x 128
```

```

15 Requested threads: 4
16 Actual threads: 4
17 Schedule: dynamic
18 Time (sec): 0.000618
19 GFLOPS: 6.791
20 Verify: PASS, max_abs_diff=0.0000000000

```

随机规模 benchmark

本节按照 benchmark 流程随机生成多组 m, n, k 。每组样本中，程序都会随机生成 A 和 B ，再计算 $C = A \times B$ 。测试线程数为 4，调度方式为 static。

表 1: 随机矩阵规模 benchmark 结果

样本	实现	m	n	k	时间 (s)	GFLOPS
1	OpenMP	458	179	1371	0.004380	51.325
1	Pthreads	458	179	1371	0.004517	49.772
2	OpenMP	1125	596	1402	0.035874	52.409
2	Pthreads	1125	596	1402	0.034081	55.166
3	OpenMP	1145	1441	1063	0.068255	51.393
3	Pthreads	1145	1441	1063	0.068131	51.486
4	OpenMP	359	197	420	0.001364	43.565
4	Pthreads	359	197	420	0.001233	48.185

从结果可以看出，随机规模下两个实现均能保持较稳定的吞吐率。较小样本（如样本 4）的 GFLOPS 低于大样本，主要原因是线程创建、调度和计时开销在小规模计算中占比更高；较大样本中计算量更充分，并行开销被摊薄，因此 GFLOPS 更稳定。OpenMP 与 Pthreads 在同一随机规模下的结果接近，说明手动实现的 `parallel_for` 能正确完成循环分解与线程执行。

实验结论

1. OpenMP 能以较少代码实现并行矩阵乘法，适合规则循环的快速并行化。
2. 对矩阵乘法这类每次迭代工作量接近的任务，静态调度通常具有较低开销和较好性能。
3. Pthreads 可以构造类似 OpenMP 的 `parallel_for` 抽象，但需要显式处理线程创建、任务划分和同步。

4. 随着线程数增加，程序总体性能提升明显，但实际加速比会受到线程管理开销、缓存行为和硬件核心数限制。